SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

---

Code of Conduct:

I **R.Pranathi(192525076)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: R. Pranathi

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. **A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF**

ANS:

Unnormalized Table (UNF)

StudentID	StudentName	Depart-ment	CourseID	CourseName	Faculty	FacultyPhone
S101	sonu	CSE	C101	DBMS	Dr. Kumar	9876543210
S101	sonu	CSE	C102	Python	Dr. Meena	9876543222
S102	pranathi	ECE	C101	DBMS	Dr. Kumar	9876543210

Normalization Issues

The table has the following problems:

- **Data Redundancy:** Student and faculty information is repeated.
- **Update Anomaly:** Changing a faculty phone number requires updating multiple rows.
- **Insertion Anomaly:** A new course cannot be added until a student enrolls.
- **Deletion Anomaly:** If the last student leaves a course, course information is lost.

First Normal Form (1NF)

Rule: Remove repeating groups and ensure every column contains atomic values.

Table: Student_Course

StudentID	StudentName	Department	CourseID	CourseName	Faculty	FacultyPhone
S101	sonu	CSE	C101	DBMS	Dr. Kumar	9876543210
S101	sonu	CSE	C102	Python	Dr. Meena	9876543222
S102	pranathi	ECE	C101	DBMS	Dr. Kumar	9876543210

Primary Key: **(StudentID, CourseID)**

Second Normal Form (2NF)

Rule: Remove partial dependency.

Student Table

StudentID	StudentName	Department
S101	sonu	CSE
S102	pranathi	ECE

Primary Key: **StudentID**

Course Table

CourseID	CourseName	Faculty	FacultyPhone
C101	DBMS	Dr. Kumar	9876543210
C102	Python	Dr. Meena	9876543222

Primary Key: **CourseID**

Enrollment Table

StudentID	CourseID
S101	C101
S101	C102
S102	C101

Primary Key: **(StudentID, CourseID)**

Foreign Keys:

- StudentID → Student Table
- CourseID → Course Table

Third Normal Form (3NF)

Rule: Remove transitive dependency.

Student

StudentID	StudentName	Department
S101	Rahul	CSE
S102	Priya	ECE

Faculty

FacultyID	FacultyName	FacultyPhone
F101	Dr. Kumar	9876543210
F102	Dr. Meena	9876543222

Course

CourseID	CourseName	FacultyID
C101	DBMS	F101
C102	Python	F102

Foreign Key: **FacultyID → Faculty**

Enrollment

StudentID	CourseID
S101	C101
S101	C102
S102	C101

Foreign Keys:

- StudentID → Student
- CourseID → Course

Final 3NF Schema

Student(StudentID, StudentName, Department)
Faculty(FacultyID, FacultyName, FacultyPhone)
Course(CourseID, CourseName, FacultyID)
Enrollment(StudentID, CourseID)

2. **A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach**

ANS:

```
mysql> CREATE DATABASE CompanyDB;
Query OK, 1 row affected (0.08 sec)

mysql> USE CompanyDB;
Database changed
mysql> CREATE TABLE Employee (EmpID INT PRIMARY KEY,EmpName VARCHAR(50),Designation VARCHAR(50));
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE Department (DeptID INT PRIMARY KEY,DeptName VARCHAR(50));
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Employee_Department (EmpID INT,DeptID INT,PRIMARY KEY (EmpID, DeptID),FOREIGN KEY (EmpID) REFERENCES Employee(EmpID),FOREIGN KEY (DeptID
) REFERENCES Department(DeptID));
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Employee VALUES(101,'Rahul','Manager'),(102,'Priya','Developer'),(103,'Arjun','Analyst'),(104,'Sneha','HR');
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Department VALUES(1,'HR'),(2,'Finance'),(3,'IT'),(4,'Marketing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Employee_Department VALUES(101,1),(101,3),(102,3),(103,2),(103,4),(104,1);
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT e.EmpID,e.EmpName,COUNT(ed.DeptID) AS TotalDepartments FROM Employee e JOIN Employee_Department ed ON e.EmpID = ed.EmpID GROUP BY e.EmpID, e.E
mpName HAVING COUNT(ed.DeptID) > 1;
+-------+---------+------------------+
| EmpID | EmpName | TotalDepartments |
+-------+---------+------------------+
|   101 | Rahul   |                2 |
|   103 | Arjun   |                2 |
+-------+---------+------------------+
2 rows in set (0.01 sec)
```

The query uses an INNER JOIN to combine the Employee and Employee_Department tables based on the common EmpID, allowing employee details to be linked with their department assignments. The GROUP BY clause groups all department records for each employee, while the COUNT() aggregate function calculates how many departments each employee is associated with. The HAVING COUNT(DeptID) > 1 condition filters the results to display only employees who work in more than one department. This approach is efficient, easy to understand, avoids duplicate records, and correctly represents the many-to-many relationship between employees and departments in a normalized database. It also provides accurate and scalable results for organizations with large datasets.

3. **Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose** it.

ANS:

Given Relation

R(StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

Functional Dependencies (FDs)

1. StudentID → StudentName
2. CourseID → CourseName, FacultyID
3. FacultyID → FacultyName

Determine the Candidate Key

attribute(s) that can determine all other attributes.

- StudentID determines only StudentName.
- CourseID determines CourseName and FacultyID.
- FacultyID determines FacultyName.

Therefore:
(StudentID, CourseID) determines:

- StudentName (from StudentID)
- CourseName (from CourseID)
- FacultyID (from CourseID)
- FacultyName (from FacultyID)

Candidate Key = (StudentID, CourseID)

Check BCNF
BCNF Rule:
For every functional dependency X → Y, X must be a superkey.

Functional Dependency	Is Left Side a Superkey?	BCNF
StudentID → StudentName	No	Violates BCNF
CourseID → CourseName, FacultyID	No	Violates BCNF
FacultyID → FacultyName	No	Violates BCNF

Since the determinants (StudentID, CourseID, and FacultyID) are not superkeys, the relation does not satisfy BCNF.

BCNF Decomposition
Student Table

StudentID (PK)	StudentName
S101	Rahul
S102	Priya

Faculty Table

FacultyID (PK)	FacultyName
F101	Dr. Kumar
F102	Dr. Meena

Course Table

CourseID (PK)	CourseName	FacultyID (FK)
C101	DBMS	F101
C102	Python	F102

Enrollment Table

StudentID (FK)	CourseID (FK)
S101	C101
S101	C102
S102	C101

Primary Key: (StudentID, CourseID)
Final BCNF Schema
Student(StudentID, StudentName)
Faculty(FacultyID, FacultyName)
Course(CourseID, CourseName, FacultyID)
Enrollment(StudentID, CourseID)

Conclusion

The original relation violated BCNF because some functional dependencies had determinants that were not superkeys. By decomposing the relation into Student, Faculty, Course, and Enrollment tables, every determinant becomes a key in its respective relation. This removes redundancy, prevents update anomalies, and ensures the database satisfies Boyce-Codd Normal Form (BCNF).

4. **A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.**

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE BankingDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE BankingDB;
Database changed
mysql> CREATE TABLE Customer (CustomerID INT PRIMARY KEY,CustomerName VARCHAR(50) NOT NULL,AccountNumber VARCHAR(20) UNIQUE,Balance DECIMAL(10,2) CHECK (Bal
ance >= 0),Branch VARCHAR(50),Phone VARCHAR(15) UNIQUE);
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO Customer VALUES(101,'Rahul','ACC1001',25000.00,'Chennai','9876543210'),(102,'Priya','ACC1002',18000.00,'Coimbatore','9876543211'),(103,'A
rjun','ACC1003',32000.00,'Madurai','9876543212');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE VIEW Customer_View AS SELECT CustomerID,CustomerName,AccountNumber,Balance,Branch FROM Customer;
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT * FROM Customer_View;
+------------+--------------+---------------+----------+------------+
| CustomerID | CustomerName | AccountNumber | Balance  | Branch     |
+------------+--------------+---------------+----------+------------+
|        101 | Rahul        | ACC1001       | 25000.00 | Chennai    |
|        102 | Priya        | ACC1002       | 18000.00 | Coimbatore |
|        103 | Arjun        | ACC1003       | 32000.00 | Madurai    |
+------------+--------------+---------------+----------+------------+
3 rows in set (0.00 sec)
```

Constraints Used

Constraint	Purpose
PRIMARY KEY	Ensures each customer has a unique CustomerID.
NOT NULL	Prevents CustomerName from being empty.
UNIQUE	Prevents duplicate AccountNumber and Phone values.
CHECK	Ensures the account balance is never negative.

Justification of the Approach

The solution uses **database views** to provide secure access by displaying only the information required by different users while hiding sensitive data such as customer phone numbers. This improves confidentiality and reduces the risk of unauthorized access. **Integrity constraints** such as **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, and **CHECK** maintain data accuracy and consistency by preventing duplicate records, null values in mandatory fields, invalid account balances, and incorrect customer information. Together, views and constraints improve database security, enforce business rules, and maintain data integrity in the banking system.

5. **A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy**

ANS:

Unnormalized Relation (UNF)

Sales

OrderID	CustomerID	CustomerName	ProductID	ProductName	SupplierID	SupplierName	Quantity
O101	C101	Rahul	P101	Laptop	S101	Dell	2
O101	C101	Rahul	P102	Mouse	S102	Logitech	1
O102	C102	Priya	P101	Laptop	S101	Dell	1

Problems in the Schema

The table contains the following anomalies:

- **Data Redundancy:** Customer, product, and supplier details are repeated.
- **Update Anomaly:** Updating a supplier or customer name requires changes in multiple rows.
- **Insertion Anomaly:** A new product or supplier cannot be added unless an order exists.
- **Deletion Anomaly:** Deleting the last order of a product removes the product and supplier information.

First Normal Form (1NF)

Rule: Remove repeating groups and ensure all attributes contain atomic values.
The relation already satisfies 1NF because each column contains a single value.

Second Normal Form (2NF)

Rule: Remove partial dependencies.

Customer Table

CustomerID (PK)	CustomerName
C101	Rahul
C102	Priya

Supplier Table

SupplierID (PK)	SupplierName
S101	Dell
S102	Logitech

Product Table

ProductID (PK)	ProductName	SupplierID (FK)
P101	Laptop	S101
P102	Mouse	S102

Sales Table

OrderID	CustomerID (FK)	ProductID (FK)	Quantity
O101	C101	P101	2
O101	C101	P102	1
O102	C102	P101	1

Third Normal Form (3NF)

The transitive dependency has been removed by separating Supplier information from Product.

Final 3NF Schema

Customer(CustomerID, CustomerName)
Supplier(SupplierID, SupplierName)
Product(ProductID, ProductName, SupplierID)
Sales(OrderID, CustomerID, ProductID, Quantity)

Final Tables

Customer

CustomerID	CustomerName
C101	Rahul
C102	Priya

Supplier

SupplierID	SupplierName
S101	Dell
S102	Logitech

Product

ProductID	ProductName	SupplierID
P101	Laptop	S101
P102	Mouse	S102

Sales

OrderID	CustomerID	ProductID	Quantity
O101	C101	P101	2
O101	C101	P102	1
O102	C102	P101	1

Conclusion

The original sales database contained redundancy and suffered from update, insertion, and deletion anomalies. After normalization to 3NF, customer, supplier, product, and sales information are stored in separate tables with proper relationships. This eliminates redundancy, improves data integrity, and makes the database easier to maintain.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.08 sec)

mysql> USE SalesDB;
Database changed
mysql> CREATE TABLE Customers ( CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), City VARCHAR(50) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Orders ( OrderID INT PRIMARY KEY, CustomerID INT, OrderDate DATE, Amount DECIMAL(10,2), FOREIGN KEY (CustomerID) REFERENCES Customers(Cu
stomerID) );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO Customers VALUES (101,'sonu','Nellore'), (102,'roops','Hyderabad'), (103,'pranathi','Madurai'), (104,'abhighna','bengaluru');
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Orders VALUES (1,101,'2025-01-10',5000), (2,101,'2025-01-15',3000), (3,101,'2025-01-20',2500), (4,102,'2025-01-12',4000), (5,103,'2025-01
-18',3500), (6,103,'2025-01-25',2000), (7,104,'2025-01-28',1500);
Query OK, 7 rows affected (0.08 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT c.CustomerID, c.CustomerName, o.OrderID, o.OrderDate, o.Amount FROM Customers c JOIN Orders o ON c.CustomerID = o.CustomerID;
+------------+--------------+---------+------------+---------+
| CustomerID | CustomerName | OrderID | OrderDate  | Amount  |
+------------+--------------+---------+------------+---------+
|        101 | sonu         |       1 | 2025-01-10 | 5000.00 |
|        101 | sonu         |       2 | 2025-01-15 | 3000.00 |
|        101 | sonu         |       3 | 2025-01-20 | 2500.00 |
|        102 | roops        |       4 | 2025-01-12 | 4000.00 |
|        103 | pranathi     |       5 | 2025-01-18 | 3500.00 |
|        103 | pranathi     |       6 | 2025-01-25 | 2000.00 |
|        104 | abhighna     |       7 | 2025-01-28 | 1500.00 |
+------------+--------------+---------+------------+---------+
7 rows in set (0.07 sec)

mysql> SELECT CustomerID, CustomerName FROM Customers WHERE CustomerID IN ( SELECT CustomerID FROM Orders GROUP BY CustomerID HAVING COUNT(OrderID) >= 3 );
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
|        101 | sonu         |
+------------+--------------+
1 row in set (0.17 sec)
```

```
mysql> SELECT CustomerID, CustomerName FROM Customers WHERE CustomerID IN ( SELECT CustomerID FROM Orders GROUP BY CustomerID HAVING COUNT(OrderID) >= 3 );
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
|        101 | sonu         |
+------------+--------------+
1 row in set (0.17 sec)

mysql> SELECT c.CustomerID, c.CustomerName, COUNT(o.OrderID) AS TotalOrders FROM Customers c JOIN Orders o ON c.CustomerID = o.CustomerID GROUP BY c.Custome
rID, c.CustomerName;
+------------+--------------+-------------+
| CustomerID | CustomerName | TotalOrders |
+------------+--------------+-------------+
|        101 | sonu         |           3 |
|        102 | roops        |           1 |
|        103 | pranathi     |           2 |
|        104 | abhighna     |           1 |
+------------+--------------+-------------+
4 rows in set (0.05 sec)
```

7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

ANS:

Given Relation

R(StudentID, StudentName, Course, Hobby)

Sample Data

StudentID	StudentName	Course	Hobby
S101	Rahul	DBMS	Cricket
S101	Rahul	DBMS	Music
S101	Rahul	Python	Cricket
S101	Rahul	Python	Music

S102	Priya	Java	Reading
S102	Priya	Java	Dance

Identify the Multi-Valued Dependencies (MVDs)

A student can enroll in **multiple courses** and can also have **multiple hobbies**, where courses and hobbies are independent of each other.

The multi-valued dependencies are:

- **StudentID →→ Course**
- **StudentID →→ Hobby**

Since these are independent multi-valued attributes, the relation contains **multi-valued dependencies**, leading to unnecessary repetition of data.

Problems in the Relation

The relation suffers from:

- **Data Redundancy:** Course and hobby information is repeated.
- **Update Anomaly:** Updating a hobby or course requires changes in multiple rows.
- **Insertion Anomaly:** Adding a new hobby requires repeating course information.
- **Deletion Anomaly:** Deleting a course may unintentionally remove hobby information.

Decompose into 4NF

Student Table

StudentID (PK)	StudentName
S101	Rahul
S102	Priya

Student_Course Table

StudentID (FK)	Course
S101	DBMS
S101	Python
S102	Java

Student_Hobby Table

StudentID (FK)	Hobby
S101	Cricket
S101	Music
S102	Reading
S102	Dance

Final 4NF Schema

Student(StudentID, StudentName)
Student_Course(StudentID, Course)
Student_Hobby(StudentID, Hobby)

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Department ( DepartmentID INT PRIMARY KEY, DepartmentName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, Email VARCHAR(100) UNIQUE, Age INT CHECK (Age >= 18), DepartmentI
D INT, FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID) );
Query OK, 0 rows affected (0.12 sec)

mysql> INSERT INTO Department VALUES (1,'Computer Science'), (2,'Electronics'), (3,'Mechanical');
Query OK, 3 rows affected (0.09 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student VALUES (101,'pranathi','pranathi@gmail.com',20,1), (102,'sonu','sonu@gmail.com',21,2), (103,'roops','roops@gmail.com',22,1);
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------------+-------------------+------+--------------+
| StudentID | StudentName | Email             | Age  | DepartmentID |
+-----------+-------------+-------------------+------+--------------+
|       101 | pranathi    | pranathi@gmail.com |   20 |            1 |
|       102 | sonu        | sonu@gmail.com     |   21 |            2 |
|       103 | roops       | roops@gmail.com    |   22 |            1 |
+-----------+-------------+-------------------+------+--------------+
3 rows in set (0.00 sec)
```

Evaluation of the Schema

The existing database schema does not enforce proper integrity constraints, which can result in inconsistent and invalid data. The following issues may occur:

- Duplicate StudentID values may be inserted.
- StudentName may be left empty (NULL).
- Duplicate email addresses can exist for multiple students.
- Invalid age values (such as negative numbers or ages below the allowed limit) may be entered.
- A student may be assigned to a DepartmentID that does not exist in the Department table, causing referential integrity problems.

These issues lead to inaccurate, inconsistent, and unreliable data.

Suggested Integrity Constraints

Constraint	**Purpose**
PRIMARY KEY	Ensures each student has a unique StudentID and prevents NULL values.
NOT NULL	Ensures mandatory fields such as StudentName and DepartmentName are not left empty.
UNIQUE	Prevents duplicate values in fields such as Email or Account Number.
CHECK	Restricts values to a valid range (e.g., Age >= 18 or Balance >= 0).
FOREIGN KEY	Ensures that DepartmentID exists in the Department table, maintaining referential integrity.
DEFAULT *(optional)*	Assigns a default value if no value is provided (e.g., Status = 'Active').

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy

ANS:

Given Relation

R(SupplierID, PartID, ProjectID)

Sample Data

SupplierID	PartID	ProjectID
S1	P1	J1
S1	P2	J1
S2	P1	J2
S2	P2	J2

This relation stores which supplier supplies which part to which project**.**

Analyze Join Dependency

A join dependency (JD) exists when a relation can be reconstructed by joining multiple smaller relations without losing information.

In this relation:

- A supplier supplies multiple parts.
- A supplier works on multiple projects.
- A part can be used in multiple projects**.**

Thus, the relation has a join dependency:

R(SupplierID, PartID, ProjectID)

can be reconstructed from:

- **R1(SupplierID, PartID)**
- **R2(SupplierID, ProjectID)**
- **R3(PartID, ProjectID)**

The original relation contains redundant combinations of supplier, part, and project**.**

Problems in the Relation

The relation suffers from:

- Data Redundancy: The same supplier, part, and project combinations are stored repeatedly.
- Update Anomaly: Updating supplier, part, or project information requires changes in multiple rows.
- Insertion Anomaly: New supplier-part or supplier-project relationships cannot be added independently.
- Deletion Anomaly: Deleting one record may remove important relationship information.

Decompose into 5NF

Supplier_Part

SupplierID	PartID
S1	P1
S1	P2
S2	P1
S2	P2

Supplier_Project

SupplierID	ProjectID
S1	J1
S2	J2

Part_Project

PartID	ProjectID
P1	J1
P2	J1
P1	J2
P2	J2

Final 5NF Schema

Supplier_Part(SupplierID, PartID)
Supplier_Project(SupplierID, ProjectID)
Part_Project(PartID, ProjectID)

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

ANS:

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.09 sec)

mysql> USE SalesDB;
Database changed
mysql> CREATE TABLE Customers ( CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), City VARCHAR(50) );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Orders ( OrderID INT PRIMARY KEY, CustomerID INT, OrderDate DATE, Amount DECIMAL(10,2), FOREIGN KEY (CustomerID) REFERENCES Customers(Cu
stomerID) );
Query OK, 0 rows affected (0.18 sec)

mysql> INSERT INTO Customers VALUES (101,'sonu','nellore'), (102,'pranathi','chennai'), (103,'roops','Madurai');
Query OK, 3 rows affected (0.10 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Orders VALUES (1,101,'2025-01-10',5000), (2,101,'2025-01-15',3000), (3,102,'2025-01-18',4500), (4,103,'2025-01-20',6000);
Query OK, 4 rows affected (0.09 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Orders WHERE CustomerID IN ( SELECT CustomerID FROM Customers WHERE City='Chennai' );
+---------+------------+------------+---------+
| OrderID | CustomerID | OrderDate  | Amount  |
+---------+------------+------------+---------+
|       3 |        102 | 2025-01-18 | 4500.00 |
+---------+------------+------------+---------+
1 row in set (0.05 sec)

mysql> SELECT o.OrderID, c.CustomerName, c.City, o.OrderDate, o.Amount FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID WHERE c.City = 'Chennai
';
+---------+--------------+---------+------------+---------+
| OrderID | CustomerName | City    | OrderDate  | Amount  |
+---------+--------------+---------+------------+---------+
|       3 | pranathi     | chennai | 2025-01-18 | 4500.00 |
+---------+--------------+---------+------------+---------+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_customer ON Orders(CustomerID);
Query OK, 0 rows affected (0.13 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Analysis of the Queries

The original query uses a **subquery** with the IN operator to retrieve customer records. Although it returns the correct result, it can be less efficient when the database contains a large number of records because the subquery must first retrieve matching CustomerID values before the outer query is executed.

The optimized query replaces the subquery with an **INNER JOIN**, allowing the database to retrieve matching records directly from both tables. Most database management systems optimize JOIN operations more effectively than nested subqueries, resulting in faster

execution and better performance.
An **index** is created on the CustomerID column of the Orders table. The index speeds up searching, filtering, and joining by allowing the database engine to locate matching records quickly instead of scanning the entire table.

Optimization Techniques Used

1. **JOIN**
    - Replaced the nested subquery with an INNER JOIN.
    - Retrieves related data more efficiently.

2. **Indexing**
    - Created an index on CustomerID.
    - Improves search and join performance.

3. **Query Restructuring**
    - Simplified the SQL statement.
    - Reduced unnecessary processing.
    - Improved readability and scalability.